ЛАБОРАТОРНА РОБОТА №3
НАВЧАННЯ НЕЙРОННОЇ МЕРЕЖІ ДЛЯ ВИЯВЛЕННЯ КІБЕРІНЦИДЕНТІВ

Мета роботи

Метою лабораторної роботи є ознайомлення студентів із практичним процесом побудови, навчання та оцінювання нейронних мереж для задач багатокласової класифікації у сфері інформаційної безпеки. Студенти повинні ознайомитися із бібліотеками TensorFlow та Keras для створення моделі, а також оцінювати результати роботи моделі за допомогою основних метрик: точності, F1, матриці помилок та кривих ROC-AUC. Додатково робота спрямована на розвиток навичок обробки результатів навчання, візуалізації ключових характеристик роботи моделі та документування передбачень, включаючи ідентифікацію неправильно класифікованих випадків.

Теоретичні відомості:

Бібліотеки Tensor Flow / Keras, PyTorch та sklearn

Бібліотека **TensorFlow** для середовища Python є однією з провідних платформ для розробки та навчання моделей машинного навчання і глибоких нейронних мереж. Вона забезпечує гнучке середовище для створення як простих, так і складних моделей, дозволяючи ефективно працювати з великими об’ємами даних та підтримувати апаратне прискорення на графічних процесорах.

Бібліотека **Keras** спочатку виникла як окрема високорівнева бібліотека для швидкого прототипування нейронних мереж, а сьогодні є інтегрованою частиною TensorFlow, надаючи зручний інтерфейс для побудови моделей. Головна різниця між ними полягає у рівні абстракції: TensorFlow забезпечує низькорівневий контроль і більшу гнучкість у створенні складних архітектур, тоді як Keras дозволяє легко і швидко конструювати моделі за допомогою простого та зрозумілого синтаксису.

Головною альтернативою поєднанню Keras/TensorFlow є бібліотека **PyTorch**, яка буде розглядатися в подальших лабораторних роботах. У порівнянні з PyTorch, Keras/TensorFlow часто вважаються легшим у застосуванні рішенням, що має великий набір готових функцій, підтримку сервісів для розгортання моделей і сумісність із різними платформами.

PyTorch, натомість, більше орієнтований на наукові дослідження, швидке прототипування та гнучку динамічну графову структуру, що дозволяє змінювати архітектуру моделі «на ходу».

Процес навчання нейронних мереж у Keras передбачає визначення архітектури моделі, вибір функції втрат, оптимізатора та метрик оцінювання. Користувач створює шари мережі у вигляді послідовності або складнішої графової структури, задаючи кількість нейронів, функції активації та додаткові елементи. Після компіляції моделі, яка включає вибір алгоритму оптимізації (наприклад, **Adam**, **SGD**), визначення функції втрат (наприклад, категоріальна крос-ентропія для багатокласової класифікації) та метрик, модель тренується на даних за допомогою методу *fit*.

Процес навчання відбувається ітераційно у вигляді епох: за кожну епоху модель проходить по всіх даних певними порціями (батчами), обчислює градієнти, оновлює ваги та оцінює помилку. Параметри, які можна задавати, включають кількість епох, розмір батча, частину даних для валідації та частоту виведення інформації про навчання.

Процес **валідації** є важливою частиною навчання нейронної мережі і використовується для контролю якості моделі під час її тренування. Під час валідації модель перевіряється на окремій підмножині даних (validation set), яка не використовується для оновлення ваг, але бере участь у

проміжній оцінці після кожної епохи навчання. Це дозволяє відслідковувати явище перенавчання (overfitting) і своєчасно коригувати гіперпараметри моделі.

На відміну від валідації, тестування виконується один раз після завершення навчання моделі і має на меті фінальну незалежну оцінку її узагальнювальної здатності. Тестовий набір даних також не використовується під час навчання і налаштування моделі, однак результати тестування вважаються остаточними і відображають реальну якість моделі на нових, раніше не бачених даних. Таким чином, валідація використовується для налаштування і контролю процесу навчання, тоді як тестування — для фінальної перевірки отриманої моделі.

Keras автоматично обробляє більшість повторюваних кроків, забезпечуючи швидке та зручне навчання моделей, при цьому зберігаючи можливість налаштовувати глибокі параметри наприклад, ранню зупинку для запобігання перенавчанню.

Scikit-learn (sklearn) — це одна з найпопулярніших бібліотек для машинного навчання на Python, яка надає широкий набір інструментів для класифікації, регресії, кластеризації, попередньої обробки даних та, особливо, оцінювання якості моделей. Вона не використовується для побудови глибоких нейронних мереж, як TensorFlow / Keras або PyTorch, але є стандартом де-факто для обчислення метрик та аналізу результатів у задачах машинного навчання.

Перевага sklearn у контексті оцінювання моделей полягає в тому, що вона надає готові, уніфіковані та перевірені реалізації основних метрик, таких як accuracy, precision, recall, F1-міра, криві ROC-AUC, а також інструменти для побудови матриці помилок та детальних звітів класифікації. Важливо, що ці функції працюють безпосередньо з масивами істинних та передбачених значень, що робить їх універсальними і незалежними від того, якою бібліотекою була побудована модель.

На відміну від Keras, який орієнтований переважно на процес навчання нейронної мережі і надає лише базові метрики в межах тренування, sklearn дозволяє виконувати більш глибокий постаналіз моделі. Наприклад, ROC-AUC у багатокласовій постановці, детальна F1-міра для кожного класу або повноцінна матриця помилок реалізуються в sklearn значно простіше та прозоріше. Саме тому в практичних роботах із машинного навчання прийнято використовувати Keras або PyTorch для навчання моделей, а sklearn — для їх детальної оцінки та інтерпретації результатів.

Огляд методів Tensorflow / Keras для машинного навчання

Основна ідея Keras полягає у тому, щоб дозволити користувачу описувати модель як послідовність шарів або як граф обчислень. Найпростішим варіантом є послідовна модель `Sequential`, у якій шари додаються один за одним. Наприклад, створення простої нейронної мережі виглядає наступним чином: модель ініціалізується як `model = keras.Sequential()`, після чого до неї додаються шари за допомогою методу `add()`. Типовий шар повнозв'язної мережі задається як `Dense`, де можна вказати кількість нейронів та функцію активації, наприклад `Dense(128, activation='relu')`.

Після визначення архітектури моделі виконується її компіляція за допомогою методу `compile()`, у якому задаються ключові гіперпараметри навчання. До них належать оптимізатор (наприклад, `adam` або `sgd`), функція втрат (наприклад, `sparse_categorical_crossentropy` для задач багатокласової класифікації) та метрики оцінювання, такі як `accuracy`. Саме на цьому етапі модель отримує інструкції щодо того, як вона буде навчатися і як оцінювати свою якість.

Процес навчання виконується методом `fit()`, який запускає ітераційне оновлення ваг нейронної мережі. Основними параметрами цього процесу є вхідні дані `X_train` і `y_train`, кількість епох (`epochs`), розмір батча (`batch_size`) та частка даних для валідації (`validation_split`). Епоха означає повний прохід моделі по всьому набору даних, тоді як батч визначає кількість прикладів, які

обробляються за одну ітерацію перед оновленням ваг. Таким чином, одна епоха складається з багатьох ітерацій, кількість яких залежить від розміру батча.

Після навчання модель може бути використана для передбачення нових даних за допомогою методу `predict()`, який повертає ймовірності належності до кожного класу. Для отримання фінального класу зазвичай застосовується операція `argmax`, яка вибирає клас із найбільшою ймовірністю.

Таким чином, Keras забезпечує інтуїтивно зрозумілий і компактний інтерфейс для побудови нейронних мереж, приховуючи складність низькорівневих обчислень TensorFlow, але при цьому залишаючи можливість детального налаштування гіперпараметрів та архітектури моделі.

Опис програмної реалізації у Python

Приклад програмної реалізації навчання нейронної мережі із використанням вищеописаних методів зображений на рис. 1. У даному прикладі була створена нейронна мережа із наступною структурою і параметрами:

1. Вхідний шар (задається неявно через `input_shape`).
2. Два приховані шари (задаються через `layers.Dense`, активаційні функції — ReLU (`activation='relu'`)). Кількість нейронів на першому з них — 128, на другому — 64.
3. Вихідний шар (останній із заданих через `layers.Dense` шар, активаційна функція — Softmax (`activation='softmax'`)).
4. Оптимізатор Adam (`optimizer='adam'`) та підрахунок втрат через категорійну крос-ентропію (`sparse_categorical_crossentropy`).
5. Метрика точності (`metrics=['accuracy']`). Зазвичай вираховується через бібліотеку sklearn, тому приклад виключно демонстраційний.
6. Кількість епох — 20 (`epochs=20`).
7. Розмір батчу — 64 (`batch_ssize=64`).
8. Валідаційна вибірка — 20% (`validation_split=0.2`).

```python
Keras.py > ...
# Імпорт бібліотек Tensorflow/Keras
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

# Створення моделі
model = keras.Sequential([
    layers.Dense(128, activation='relu', input_shape=(input_dim,)),
    layers.Dropout(0.3),
    layers.Dense(64, activation='relu'),
    layers.Dropout(0.3),
    layers.Dense(num_classes, activation='softmax')
])

# Компіляція моделі (налаштування процесу навчання)
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

# Навчання моделі
history = model.fit(
    X_train,
    y_train,
    epochs=20,
    batch_size=64,
    validation_split=0.2
)

# Передбачення класів
y_pred = model.predict(X_test)
y_pred_classes = tf.argmax(y_pred, axis=1)
```

Рис. 1. Реалізація навчання нейронної мережі із використанням бібліотеки Keras

Метрики оцінювання якості роботи нейронної мережі

Після навчання моделі необхідно оцінити її якість за допомогою метрик.

Загальна **точність** (accuracy) відображає частку правильно передбачених прикладів у *загальному* наборі даних.

На відміну від точності, що розглядає загальну кількість мережевих подій, виділяють також дві характеристики, які розраховуються відносно класу *атак*: **прецизійність** (precision) та повнота (recall).

Прецизійність (precision) визначає частоту правильних передбачень «атака» відносно загальної кількості передбачень «атака».

Повнота (recall) визначає частоту правильних передбачень «атака» відносно справжньої кількості атак.

Розберемо конкретний приклад.

Кількість мережевих подій — 200.

Кількість справжніх атак — 100.

Кількість передбачень «атака» — 120.
Кількість правильних передбачень «атака» — 80.

Виходячи із цих цифр:

1. Справжньопозитивні передбачення, (True Positive, TP) = 80.
2. Пропущені атаки (False Negative, FN) = 100 – 80 = 20.
3. Хибнопозитивні передбачення (False Positive, FP) = 120 – 80 = 40.
4. Нормальні події (True Negative, TN) = Нормальні події – Хибнопозитивні передбачення; кількість нормальних подій = 200 – 100 = 100, з яких 40 модель назвала «атакою» => TN = 100 – 40 = 60.

Таким чином:

1. Точність (Accuracy) = (80 + 60) / 200 = 140 / 200 = 70%;
2. Прецизійність (Precision) = 80 / 120 = 66,7% ;
3. Повнота (Recall) = 80 / 100 = 80%.

Метрика **F1** поєднує прецизійність і повноту у гармонійне середнє, що дозволяє краще оцінювати модель при нерівномірному розподілі класів.

Матриця помилок (confusion matrix) наочно показує кількість правильних і неправильних передбачень для кожного класу, дозволяючи виявити, які класи модель плутає найчастіше.

Крива **ROC** (Receiver Operating Characteristic) та пов'язана із нею метрика **AUC** (Area Under the Curve) демонструє здатність моделі відділяти один клас від іншого, враховуючи баланс між справжньопозитивними і хибнопозитивними спрацюваннями.

Хід роботи

Виконання лабораторної роботи №3 передбачає володіння деякими навичками засвоєними у лабораторній роботі №1 та №2, а саме основам використання GitHub та Google Colab. Відповідно, детально розписуватися вони не будуть. Етапи виконання лабораторної роботи №3 наступні:

1. Перейти на **GitHub**-репозиторій за посиланням: **github.com**
2. Завантажити файл *Lab3 - Google Colab.ipynb.*
3. Підготувати отримані у лабораторній роботі №2 оброблений та тренувальні датасети *train_processed.csv* та *test_processed.csv*.
4. Перейти на платформу **Google Colab**: **посилання**.
5. На сторінці, що відкрилася, вибрати пункт «Завантажити» і або знайти файл *Lab3 - Google Colab.ipynb* на вашому ПК, або перетягнути його у саме вікно.
6. У реалізації Python, що відкрилася, у пункті «1. Введіть своє прізвище та ім'я, а також номер варіанту» ввести необхідні дані, наприклад:

```
student_name = "Костенко Уляна"
variant_number = 21
```

7. Вище знайти кнопку «Виконати всі», почекати деякий час, доки внизу блоку «2. Запустіть код, натисніть кнопку "Вибрати файли" та виберіть тренувальний та тестовий датасети, отримані в результаті ЛР2» з'явиться кнопка «Вибрати файли», натиснути її та вибрати обидва датасети *train_processed.csv* та *test_processed.csv*.
8. Дочекатися, поки датасети завантажаться.

9. Дочекатися, доки код реалізації буде виконано та внизу блоку «3. Аналіз датасетів, навчання нейронної мережі, передбачення класів, розрахунок метрик, їхня графічна візуалізація та генерація .png та .csv файлів» з'явиться графічна візуалізація метрик якості. Внаслідок виконання цього блоку відбулися наступні процеси:

9.1. Навчання нейронної мережі на тренувальному датасеті *train_processed.csv* (з використанням валідаційної вибірки).

9.2. Подальше тестування отриманої моделі на датасеті *test_processed.csv*. Відповідно до даних цього датасету були зроблені передбачення щодо класу мережевих подій в кожному його рядку.

9.3. Порівняння передбачень моделі зі справжніми класами подій та підрахунок метрик якості нейронної мережі.

9.4. Створення додаткових .csv-файлів (справжні назви файлів включають прізвище та ім'я студента, а також номер варіанту):

a. Файл *prediction_Прізвище_Ім'я_Варіант_X.csv* — копія *test_processed.csv*, у якому була створена додаткова колонка зі зробленими передбаченнями, що дає змогу порівняти їх зі справжніми класами.

b. Файл *misclassified_Прізвище_Ім'я_Варіант_X.csv* — вибірка рядків *predictions.csv* із неправильними передбаченнями.

9.5. Візуалізація метрик якості та збереження графіків у файл *ЛР3_Прізвище_Ім'я_Варіант_X.png*.

10. Зліва натиснути на іконку «Файли», у новому вікні знайти та завантажити на свій ПК наступні файли:

10.1. Файл *ЛР3_Прізвище_Ім'я_Варіант_X.png*.

10.2. Файл *prediction_Прізвище_Ім'я_Варіант_X.csv*

10.3. Файл *misclassified_Прізвище_Ім'я_Варіант_X.csv*.

11. Завантажити у завдання лабораторної роботи №3 у Google Classroom наступні документи:

11.1. Файл *ЛР3_Прізвище_Ім'я_Варіант_X.png*.

11.2. Файл *prediction_Прізвище_Ім'я_Варіант_X.csv*

11.3. Файл *misclassified_Прізвище_Ім'я_Варіант_X.csv*.

11.4. Звіт з лабораторної роботи у форматі *.docx,* що включатиме детальний опис проведеної роботи та етапів її виконання, а також отриманих результатів. Проаналізувати відповідно до отриманої візуалізації результати обробки даних та зробити наступні висновки:

11.4.1. Проаналізуйте код файлу *Lab3 - Google Colab.ipynb*. Дайте відповіді щодо: а) кількості шарів створеної нейронної мережі; б) кількості нейронів на прихованих шарах; в) використаних активаційних функцій; г) кількості епох; ґ) розміру батчу; д) розмір валідаційної вибірки.

11.4.2. Проаналізуйте файли *prediction_Прізвище_Ім'я_Варіант_X.csv* та *misclassified_Прізвище_Ім'я_Варіант_X.csv*. Скільки із загальної кількості передбачень було зроблено невірних?

11.4.3. Проаналізуйте метрики якості з файлу *ЛР3_Прізвище_Ім'я_Варіант_X.png*. Як змінювались втрати і точність мережі зі

збільшенням номеру епохи? Як Ви можете охарактеризувати дані матриці помилок та ROC-кривої?

11.4.4. Ваша оціночна характеристика ефективності діяльності нейронної мережі.

12. Звіт закінчити відповідями на зазначені нижче питання для самоконтролю.

Висновки

У ході виконання лабораторної роботи було розглянуто повний цикл побудови та дослідження нейронної мережі для задачі багатокласової класифікації мережевих подій. Було здійснено завантаження та підготовку попередньо оброблених даних, побудовано модель нейронної мережі з використанням бібліотеки Keras, а також виконано її навчання із застосуванням валідаційної вибірки для контролю якості.

У процесі роботи також відбулося ознайомлення з основними етапами налаштування моделі, зокрема вибором архітектури, функцій активації, оптимізатора та гіперпараметрів навчання, таких як кількість епох і розмір батча. Було проведено оцінювання якості моделі на тестовому наборі даних із використанням ключових метрик, включаючи загальну точність, F1-міру, матрицю помилок та ROC-AUC, що дозволило отримати комплексне уявлення про ефективність класифікації.

Особливу увагу було приділено аналізу результатів, зокрема виявленню помилкових передбачень та інтерпретації роботи моделі для окремих класів. Побудова графіків процесу навчання, матриці помилок і ROC-кривих дозволила наочно оцінити поведінку моделі та її здатність розрізняти класи мережевих подій.

Отримані результати підтверджують, що використання нейронних мереж у поєднанні з сучасними інструментами машинного навчання дозволяє ефективно вирішувати задачі класифікації у сфері інформаційної безпеки. Виконання лабораторної роботи сприяло формуванню практичних навичок роботи з бібліотеками TensorFlow/Keras та scikit-learn, а також розвитку вміння аналізувати та інтерпретувати результати роботи моделей машинного навчання.

Питання для самоконтролю

1. Що таке бібліотеки TensorFlow і Keras і чим вони відрізняються один від одного?
2. Чому Keras вважають високорівневою бібліотекою, а TensorFlow низькорівневою платформою? Для чого потрібні оптимізатори у навчанні нейронних мереж і які приклади оптимізаторів можна використовувати?
3. Що таке бібліотека sklearn і чому для оцінювання метрик моделі часто використовується саме вона, а не Keras?
4. Що таке функція втрат (loss) і яку роль вона відіграє у процесі навчання?
5. Що таке епоха, батч та ітерація у процесі навчання нейронної мережі?
6. Що таке валідація і чим вона відрізняється від тестування?
7. Які основні метрики оцінювання якості моделі існують і що вони вимірюють?
8. Що таке точність (accuracy)?
9. Що таке прецизійність і повнота і чим ці характеристики принципово відрізняються від точності?
10. Яким чином F1 оцінює модель і чому її використовують для багатокласової класифікації?
11. Для чого потрібна матриця помилок і як вона допомагає аналізувати роботу моделі?

12. Що таке ROC-крива і AUC, і як вони інтерпретуються у контексті багатокласової класифікації?